SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I __Mathivadhani.K_(192311424)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Mathivadhani.K

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

1. Reverse Engineering ATM Withdrawal Module

A. Introduction

Reverse engineering an ATM withdrawal module involves analyzing its input-output behavior to identify the underlying testing strategy. The system accepts withdrawal details, validates the amount and balance, processes transactions, and displays appropriate messages.

B. Functional Decomposition

- Input validation: Checks whether the account number and withdrawal amount are valid.
- Balance validation: Verifies whether sufficient money is available.
- Withdrawal limit validation: Checks daily and per-transaction withdrawal limits.
- Transaction processing: Deducts the amount from the account balance.
- Error handling: Displays messages for invalid inputs or failed transactions.

C. Equivalence Class Partitioning (ECP)

The inputs are divided into valid and invalid groups.

- Valid amount: A positive amount within the permitted limit.
- Invalid amount: Zero or negative amount.
- Insufficient balance: Amount greater than the available balance.
- Limit exceeded: Amount greater than the permitted withdrawal limit.
- Invalid account: Account number does not exist or is inactive.

D. Boundary Value Analysis (BVA)

Test case	Input condition	Expected result
TC1	Valid amount within limits	Withdrawal successful
TC2	Amount = 0	Validation error
TC3	Amount just below minimum	Reject if below the permitted minimum
TC4	Amount at maximum limit	Accept if all other conditions are valid
TC5	Amount above maximum limit	Limit error
TC6	Amount greater than balance	Insufficient balance message

E. Conclusion

ECP and BVA help identify invalid inputs and errors near withdrawal limits. The reconstructed test suite improves functional coverage and ensures that incorrect withdrawals and balance updates are prevented.

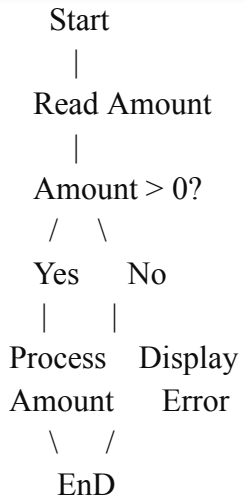
2. Reverse Engineering White Box Testing

A. Introduction

White box testing examines the internal program structure, decisions, loops, and execution paths. When the original test documentation is missing, execution traces and source code can be used to reconstruct the testing strategy.

B. Control Flow Graph (CFG)

A control flow graph represents program execution using nodes and directed edges. Consider the following logic:



The decision Amount > 0 creates two branches: true and false.

C. Independent Execution Paths

- Path 1: Start → Read amount → Condition true → Process amount → End.
- Path 2: Start → Read amount → Condition false → Display error → End.

Independent paths are execution paths that introduce at least one new edge compared with previously selected paths.

D. Test Coverage Criteria

- Statement coverage: Ensures every executable statement runs at least once.
- Branch coverage: Ensures every decision outcome is tested.
- Path coverage: Tests independent execution paths.
- Loop testing: Checks zero iterations, one iteration, and multiple iterations where applicable.

E. Conclusion

CFG construction and path testing help identify missing branches and untested logic. A combination of statement, branch, and path coverage provides stronger confidence in the correctness of the program.

3. Reverse Engineering Login and Transaction System

A. Introduction

A web application login and transaction system contains authentication, session management, transaction validation, and database operations. Defect logs can reveal weaknesses in the original testing strategy, such as missed invalid inputs or untested decision branches.

B. Functional Decomposition

- Login module: Validates usernames and passwords.
- Authentication module: Confirms user identity.
- Session module: Maintains login status and manages session expiration.
- Transaction module: Validates amount and account balance.
- Database module: Updates transaction records and balances.
- Error-handling module: Displays suitable failure messages.

C. Black Box Testing Techniques

- Equivalence Class Partitioning: Divide credentials into valid, invalid, and empty inputs.
- Boundary Value Analysis: Test minimum and maximum password lengths and transaction limits.
- Negative testing: Enter incorrect credentials, invalid amounts, and unauthorized requests.

D. White Box Testing Techniques

- Branch coverage: Test both successful and unsuccessful authentication decisions.
- Path testing: Cover successful login, failed login, valid transaction, and rejected transaction paths.
- Condition testing: Check individual conditions, such as valid credentials and sufficient balance.

Test case	Input	Expected result
TC1	Valid username and password	Login successful
TC2	Incorrect password	Login rejected
TC3	Empty credentials	Validation error
TC4	Valid transaction amount	Transaction processed
TC5	Amount exceeds balance	Transaction rejected

TC6	Expired session	Reauthentication required

E. Conclusion

Integrating black box and white box testing improves functional coverage and helps identify authentication, transaction, and database errors. Security tests should also cover unauthorized access and duplicate transaction requests.

4. Reverse Engineering Inventory Management System

A. Introduction

An inventory management system maintains product quantities and generates alerts when stock reaches a predefined reorder level. Successful and failed transaction logs can be analyzed to infer stock validation rules and hidden boundary conditions.

B. Functional Decomposition

- Product identification: Checks whether the product exists.
- Quantity validation: Ensures that the quantity is valid.
- Stock update: Adds or subtracts stock from the current quantity.
- Threshold checking: Compares stock against the reorder level.
- Alert generation: Notifies users when stock is low.
- Database update: Saves the revised inventory information.

C. Equivalence Class Partitioning

- Valid quantity: Positive quantity within the permitted range.
- Zero quantity: No stock change or rejection, depending on the requirements.
- Negative quantity: Invalid input unless the operation explicitly represents a stock reduction.
- Excessive quantity: Quantity exceeding the maximum permitted value.
- Invalid product ID: Product does not exist.
- Low-stock condition: Updated stock is at or below the reorder threshold.

D. Reconstructed Test Cases

Test case	Condition	Expected result
TC1	Add a valid quantity	Stock increases correctly
TC2	Enter a negative quantity	Invalid input rejected
TC3	Update stock to the reorder level	Low-stock alert generated

TC4	Update stock above the reorder level	No low-stock alert
TC5	Use an invalid product ID	Error displayed
TC6	Quantity exceeds maximum capacity	Update rejected

Boundary testing should include values immediately below, equal to, and above the reorder threshold and quantity limits.

E. Conclusion

ECP and BVA provide systematic coverage of valid and invalid inventory updates. Testing alert thresholds and database consistency helps prevent incorrect stock values and missed low-stock notifications.

5. Reverse Engineering a Library Management EXE Using Execution Profiles

A. Introduction

A compiled library management executable can be analyzed using CPU and memory profiles even when its source code is unavailable. Execution spikes may indicate repeated loops, expensive database operations, or memory-intensive processing. However, profiling alone cannot prove the exact internal control flow.

B. Functional Decomposition

- Book search: Searches for books using an ID, title, or author.
- Availability checking: Determines whether a book can be issued.
- Issue processing: Records the issue transaction.
- Return processing: Updates the book's availability.
- Fine calculation: Calculates overdue charges.
- Database management: Stores and retrieves library records.

C. Conceptual Control Flow Graph



```
        |
    Available?
     /    \
   Yes     No
    |      |
 Issue Book  Display Error
    |
Update Database
    |
   End
```

This conceptual CFG identifies important decisions that can be tested independently.

D. Path Testing and Resource Analysis

- Path 1: Book not found; error message displayed.
- Path 2: Book found and available; issue transaction succeeds.
- Path 3: Book found but unavailable; issue request is rejected.
- Path 4: Database update fails; error is handled without corrupting records.
- Stress path: Repeatedly search, issue, and return books using large datasets.

During stress testing, monitor CPU usage, memory consumption, response time, and application crashes. Memory profiling should also check for leaks after repeated operations.

E. Conclusion

Combining conceptual CFG testing with profiling helps identify logical errors and resource-intensive operations. Stress testing verifies that the application remains stable and maintains correct book records under heavy workloads.

6. State Transition Testing of Mobile Payment Gateway

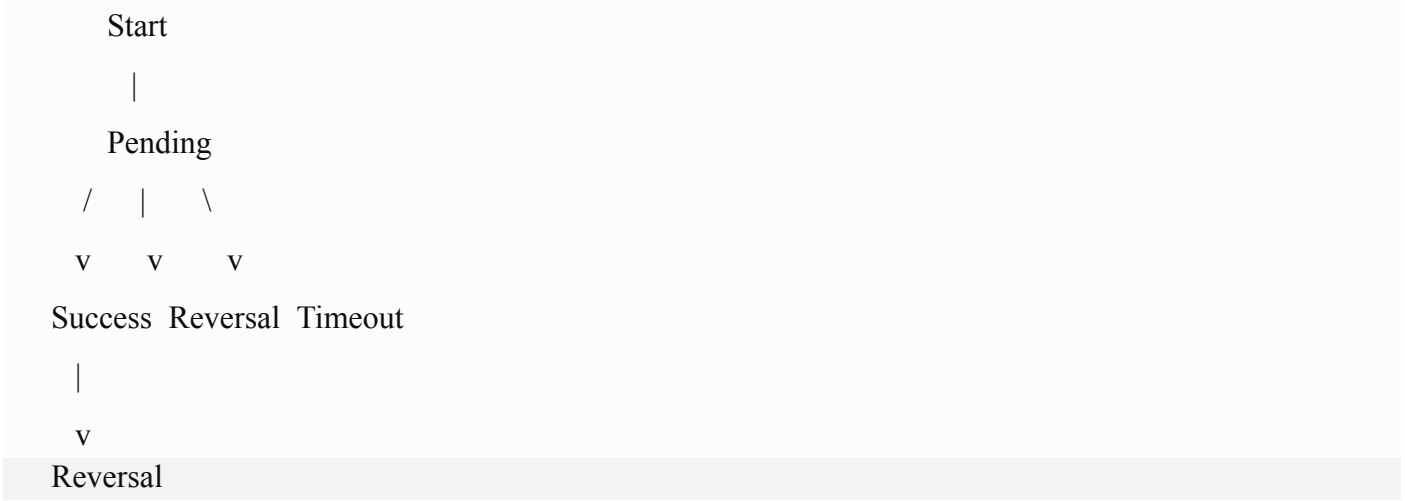
A. Introduction

A mobile payment gateway processes transactions through states such as Pending, Success, Reversal, and Timeout. State transition testing verifies that each event moves the transaction into a valid state and that illegal transitions are handled safely.

B. Identification of States

- Pending: The transaction has started but is not yet confirmed.
- Success: The payment has been confirmed.
- Reversal: A previously initiated or completed payment is reversed according to the supported business rules.
- Timeout: The expected response was not received within the permitted time.

C. State Transition Diagram



The exact legal transitions must be confirmed from the API specification. For example, a successful payment may support a later refund or reversal, while an ordinary duplicate success callback must not create a second debit.

D. State Transition Test Cases

Initial state	Event	Expected result
Pending	Valid payment confirmation	Success
Pending	Valid cancellation request	Reversal or cancellation as specified
Pending	Response timeout	Timeout or pending reconciliation
Success	Authorized reversal request	Reversal
Success	Duplicate success callback	No duplicate debit
Timeout	Late success response	Reconcile status without duplicate payment
Reversal	Duplicate reversal request	No duplicate refund

Testing should also validate transaction IDs, timestamps, response codes, and idempotency keys in JSON responses.

E. Conclusion

State transition testing ensures correct transaction status changes and reliable error handling. Covering both legal and illegal transitions helps prevent duplicate payments, incorrect reversals, and inconsistent transaction records.

7. Fuzzing and Boundary Value Analysis in Encrypted Communication

A. Introduction

An encrypted communication module accepts input data and produces encrypted outputs. Fuzzing and boundary value analysis can help reverse engineer input validation and error handling, especially when the internal implementation is unavailable.

B. Functional Decomposition

- Input validation: Checks the format and length of incoming data.
- Data preparation: Converts input into the required internal format.
- Encryption: Encrypts validated data using the configured cryptographic algorithm.
- Output generation: Returns the encrypted result.
- Error handling: Rejects malformed input and handles failures safely.

C. Fuzzing Technique

Fuzz testing supplies unexpected, malformed, or random input values to identify crashes and improper validation.

Examples include:

- Empty strings and missing parameters.
- Very long messages.
- Invalid character encodings.
- Unexpected data types.
- Malformed JSON or binary data.
- Repeated special characters and unexpected null bytes.

D. Boundary Value Test Cases

Test case	Input	Expected result
TC1	Minimum valid input	Correct processing
TC2	Empty input	Rejected or handled according to specification
TC3	Maximum permitted input length	Successful processing
TC4	Input one unit above maximum	Safe rejection

TC5	Malformed input	Controlled error
TC6	Repeated random inputs	No crash or memory corruption

To reduce buffer overflow and injection risks, the implementation should use bounds checking, safe memory management, strict input validation, and secure cryptographic APIs. Fuzzing helps reveal weaknesses but does not prove that the system is vulnerability-free.

E. Conclusion

Fuzzing and BVA help identify input-handling weaknesses, boundary errors, and unexpected failures. Combining these techniques with security testing improves the reliability of the encrypted communication module.

8. Improving White Box Testing Beyond Statement Coverage

A. Introduction

Statement coverage measures whether every executable statement has been executed at least once. However, 100% statement coverage does not guarantee that every branch or Boolean condition has been tested, especially in compound decision statements.

B. Example Code

```
if (age >= 18 && hasID)
   AllowEntry();
else
   DenyEntry();
```

The decision depends on two conditions: whether the person is at least 18 and whether the person has valid identification.

C. Control Flow Graph

```
        Start
          |
       Read Inputs
          |
  age >= 18 AND hasID?
       /      \
     Yes       No
      |        |
```

```
AllowEntry  DenyEntry
     \      /
        End
```

The CFG contains one decision node and two outgoing branches.

D. Independent Paths and Test Cases

Test case	Age	Has ID	Expected result
TC1	20	True	Allow entry
TC2	20	False	Deny entry
TC3	16	True	Deny entry
TC4	16	False	Deny entry

- Statement coverage: Executes both AllowEntry() and DenyEntry().
- Branch coverage: Tests true and false outcomes of the complete decision.
- Condition coverage: Makes each individual condition true and false.
- MC/DC: Demonstrates that each condition can independently affect the final decision outcome.

E. Conclusion

Statement coverage alone may miss logical defects in compound conditions. Branch coverage, condition coverage, and MC/DC provide stronger testing for decision logic and reduce the risk of incorrect calculations or authorization decisions.

9. Reverse Engineering Cloud File Synchronization

A. Introduction

A cloud-based synchronization tool transfers files between devices and cloud storage. Lost-file defect reports may indicate weaknesses in file-size handling, file-type validation, network interruption recovery, or conflict resolution.

B. Functional Decomposition

- File selection: Identifies files to synchronize.
- Metadata validation: Checks file names, types, sizes, and timestamps.
- Network transfer: Uploads or downloads file contents.
- Conflict resolution: Handles multiple versions of the same file.
- Integrity verification: Checks whether the transferred file matches the original.
- Synchronization logging: Records the result and supports recovery after failures.

C. Equivalence Class Partitioning

- File size: Empty, normal, maximum permitted, and oversized files.
- File type: Supported and unsupported formats.
- Network: Stable, slow, interrupted, and unavailable.
- File state: New, modified, deleted, and conflicting versions.
- Access permissions: Authorized and unauthorized files.

D. Reconstructed Test Cases

Test case	Condition	Expected result
TC1	Small file on stable network	Successful synchronization
TC2	Empty file	Correct handling
TC3	Maximum permitted file size	Successful transfer
TC4	File exceeds size limit	Safe rejection
TC5	Network interruption during transfer	Retry or resume without corruption
TC6	Two devices modify the same file	Conflict handled according to policy
TC7	Large batch of files	No missing or corrupted files

Stress testing should synchronize many files simultaneously while varying network speed and interruption frequency. Checksums and file counts should be compared before and after synchronization.

E. Conclusion

Combining ECP, BVA, and stress testing improves coverage of file sizes, types, and network conditions. Integrity checks, retry mechanisms, and conflict testing help prevent future data loss and file corruption.

10. Decision Table Testing for E-commerce Search and Filter

A. Introduction

An e-commerce search module retrieves products using keywords, categories, price ranges, and availability conditions. By examining generated SQL queries and their results, the hidden filtering logic can be reconstructed using decision table testing.

B. Functional Decomposition

- Keyword search: Finds products matching a search term.
- Category filtering: Restricts results to selected categories.
- Price filtering: Selects products within a specified range.
- Availability filtering: Includes or excludes out-of-stock products.
- Query generation: Converts search conditions into SQL.
- Result processing: Returns matching products in the required order.

C. Decision Table

Assume the system uses three conditions: keyword match, price match, and stock availability.

Rule	Keyword matches	Price matches	In stock	Expected result
R1	Yes	Yes	Yes	Include product
R2	Yes	No	Yes	Exclude product
R3	Yes	Yes	No	Exclude if in-stock filter is enabled
R4	No	Yes	Yes	Exclude product
R5	Yes	Yes	No filter applied	Include if other conditions match

The final rule depends on whether availability filtering is optional or mandatory.

D. Reconstructed Test Strategy

- AND testing: Verify that a product must satisfy all active conditions.
- OR testing: Verify that a product matching at least one alternative condition is returned.
- NOT testing: Verify that excluded categories or products do not appear.
- Boundary testing: Test minimum and maximum prices and values just outside the range.
- Combination testing: Test empty filters, conflicting filters, multiple categories, and keyword-price combinations.

For SQL validation, verify that parameterized queries are used and that special input characters cannot alter the intended query structure.

E. Conclusion

Decision table testing provides a structured method for covering combinations of search conditions. Comparing expected results with actual SQL outputs helps detect missing filters, incorrect AND/OR logic, and unintended result inclusion.